SPRING DOCS

Spring MVC

DispatcherServlet, Controllers, Request Mapping & Validation

Java Agentic AI — Knowledge Base Reference

1. The MVC Pattern in Spring

Spring MVC implements Model-View-Controller: the **Model** carries data, the **View** renders it (JSP, Thymeleaf, or JSON for REST APIs), and the **Controller** handles incoming requests, invokes business logic, and selects the model/view to return.

2. The DispatcherServlet — Front Controller

Every incoming HTTP request first hits the **DispatcherServlet**, Spring MVC's central front controller, which orchestrates the entire request lifecycle.

- 1. Request arrives at DispatcherServlet.
- 2. HandlerMapping determines which controller method should handle it, based on URL, HTTP method, headers.
- 3. HandlerAdapter invokes the matched controller method.
- 4. The controller executes business logic and returns a Model + logical view name (or directly a response body for REST).
- 5. ViewResolver maps the logical view name to an actual view template (for traditional MVC apps).
- 6. The View renders the model into the final HTML/JSON response sent back to the client.

Note: *In Spring Boot, spring-boot-starter-web auto-configures the DispatcherServlet and an embedded Tomcat, so this entire machinery is wired up with zero XML.*

3. Controllers

```
@RestController                          // = @Controller + @ResponseBody on every method
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);
    }

    @PostMapping
    public ResponseEntity<User> createUser(@Valid @RequestBody User user) {
        User saved = userService.save(user);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
```

```
    @PutMapping("/{id}")
    public User updateUser(@PathVariable Long id, @RequestBody User user) {
        return userService.update(id, user);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
        userService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

4. Request Parameter Binding

Annotation	Purpose
@PathVariable	Binds a URI template variable, e.g. /users/{id}
@RequestParam	Binds a query parameter, e.g. ?page=1&size=10
@RequestBody	Deserializes the HTTP request body (typically JSON) into a Java object
@RequestHeader	Binds an HTTP header value
@ModelAttribute	Binds form data (or query params) to a Java object's fields
@CookieValue	Binds a cookie value

```
@GetMapping("/search")
public List<User> search(
        @RequestParam String name,
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(required = false) String sortBy) {
    return userService.search(name, page, sortBy);
}
```

5. @Controller vs @RestController

Aspect	@Controller	@RestController
Return value meaning	Logical view name (resolved by a ViewResolver)	Response body directly (serialized to JSON/XML)
Needs @ResponseBody?	Yes, per method, for JSON responses	No — implied on every method
Typical use	Server-rendered HTML pages (Thymeleaf, JSP)	REST APIs

6. Validation with Bean Validation (JSR 380)

```
public class User {
    @NotBlank(message = "Name is required")
    private String name;
```

```
    @Email
    private String email;

    @Min(18)
    @Max(120)
    private int age;
}

@PostMapping
public User create(@Valid @RequestBody User user) { // @Valid triggers validation
    return userService.save(user);
}

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>>
handleValidation(MethodArgumentNotValidException ex) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getFieldErrors()
        .forEach(err -> errors.put(err.getField(), err.getDefaultMessage()));
    return ResponseEntity.badRequest().body(errors);
}
```

7. Exception Handling

```
@RestControllerAdvice         // global exception handling across all controllers
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(ex.getMessage(),
HttpStatus.NOT_FOUND.value());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        return ResponseEntity.internalServerError().body(new ErrorResponse("Unexpected
error", 500));
    }
}
```

Note: *@RestControllerAdvice centralizes exception-to-response mapping across the whole application, avoiding repetitive try-catch blocks scattered through every controller method.*

8. Interceptors and Filters

	Layer	Typical use
	Before the DispatcherServlet, at the servlet container level	Logging, CORS, authentication tokens, request/respon
	Inside Spring MVC, around controller invocation	Auth checks, logging with access to handler method m

```
public class LoggingInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object
handler) {
        System.out.println("Request: " + req.getRequestURI());
        return true;     // false would stop the request from proceeding further
    }
}

@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new LoggingInterceptor());
    }
}
```

9. Content Negotiation

Spring MVC can serve the same endpoint as JSON, XML, or other formats based on the client's Accept header, using registered HttpMessageConverters (Jackson for JSON by default).

10. Quick Interview Recap

- What is the DispatcherServlet? Spring MVC's front controller — the single entry point that routes every incoming request to the appropriate handler and manages the whole request/response lifecycle.
- @RequestParam vs @PathVariable? @RequestParam reads query string parameters; @PathVariable reads segments embedded directly in the URI path template.
- How does @Valid trigger validation? It tells Spring to run Bean Validation (Hibernate Validator) against the annotated object before the controller method body executes, populating a BindingResult with any violations.
- What's the benefit of @RestControllerAdvice? Centralizes exception handling logic globally instead of repeating try-catch in every controller.